Fashion MNIST CNN design and results

Assignment 2 | Exercise 1 part ii

We trained a convolutional neural network to classify ten types of clothing. This report explains our model choices and the results. The loss curves show signs of overfitting from around epoch 6, even though validation and test accuracy remain fairly steady.

Data and model structure

Each input is a 28 x 28 grayscale image. We use 54,000 images for training, 6,000 for validation and 10,000 for testing. Validation helps us choose the model weights. The test set measures performance on images not used to update the weights.

We divide pixel values by 255 to put them between 0 and 1. We one-hot encode each label into ten values and add one grayscale channel before the first convolution. Seed 42 makes the data split repeatable.

Stage	Main settings	Output per image
Convolution 1	32 filters, 3 x 3, stride 1, GELU	28 x 28 x 32
Max pooling 1	2 x 2, stride 2	14 x 14 x 32
Convolution 2	64 filters, 3 x 3, stride 1, GELU	14 x 14 x 64
Max pooling 2	2 x 2, stride 2	7 x 7 x 64
Convolution 3	64 filters, 3 x 3, stride 1, GELU	7 x 7 x 64
Classifier	Flatten, Dense 128, GELU, dropout 0.3	128 values
Output	Dense 10, softmax	10 probabilities

All three convolutions use one-pixel replicate padding. The complete model has 458,570 trainable parameters.

Why we use these convolutions

Three layers and 3 x 3 kernels. We want to learn local patterns first and then combine them in deeper layers. A 3 x 3 kernel examines a position and its immediate neighbors. Deeper layers combine information from a larger area of the original image.

Stride 1. The filter moves one position at a time. We avoid reducing the image size during convolution and leave spatial reduction to pooling. This does not mean that every detail is preserved.

Why we chose the remaining values

32 then 64 then 64 filters. The first layer works on larger 28 x 28 maps. Using 32 filters instead of 64 there reduces computation and memory, with the aim of saving time. After pooling, the maps are smaller: 14 x 14 and 7 x 7. We use 64 filters in those layers to learn more feature combinations. Actual speed also depends on the number of input channels.

Replicate padding. We copy the nearest edge value into a one-pixel border. This lets the filter examine neighborhoods centered on the image boundary without inserting a zero border. It keeps the spatial size after convolution. The copied values are an assumption about the boundary, not new image information.

GELU activation. We chose GELU because it is smooth and can pass gradients for negative inputs. ReLU has a zero gradient for negative inputs. GELU may help learning in this region, but it does not guarantee better results. ReLU also supports backpropagation, and GELU gradients can still become very small.

2 x 2 max pooling. We keep the largest response in each small neighborhood. Pooling with stride 2 reduces 28 to 14 and then 14 to 7. We stop pooling at 7 x 7 to keep more spatial detail before classification. Max pooling keeps the strongest response, which is not always the most useful information.

Dense 128 and dropout 0.3. The dense layer combines the 3,136 flattened features. We retained 128 hidden units as a starting size and use GELU here too. Dropout switches off 30% of these activations during training to reduce reliance on particular features. These settings were not compared with alternatives in this run.

Ten softmax outputs. There are ten clothing classes, so the final layer produces ten probabilities. We use categorical cross-entropy because the labels are one-hot encoded. Lower loss means that the model assigns more probability to the correct answers.

Training choices

Batch size 128. Assignment 1 used Iris, which has only 150 examples in total. Here we train on 54,000 images. Increasing the batch size from 32 to 128 reduces weight updates from 1,688 to 422 per epoch. Each update averages more examples, aiming for less noisy gradients and less update overhead. Smoother curves and better accuracy are not guaranteed.

Forty epochs. We kept the epoch count from Assignment 1. This lets us see both the early improvement and what happens when training continues for longer. The full run uses all 40 epochs.

Adam with learning rate 0.001. We retained these as starting training settings. Adam adjusts updates using information from earlier gradients. We did not compare different optimizers or learning rates in this experiment.

The model with the lowest validation loss is kept in memory. Test values are recorded after every epoch for the graphs, but they do not update the weights or choose the best model. Seed 42 is also used for training repeatability.

Learning curves and overfitting

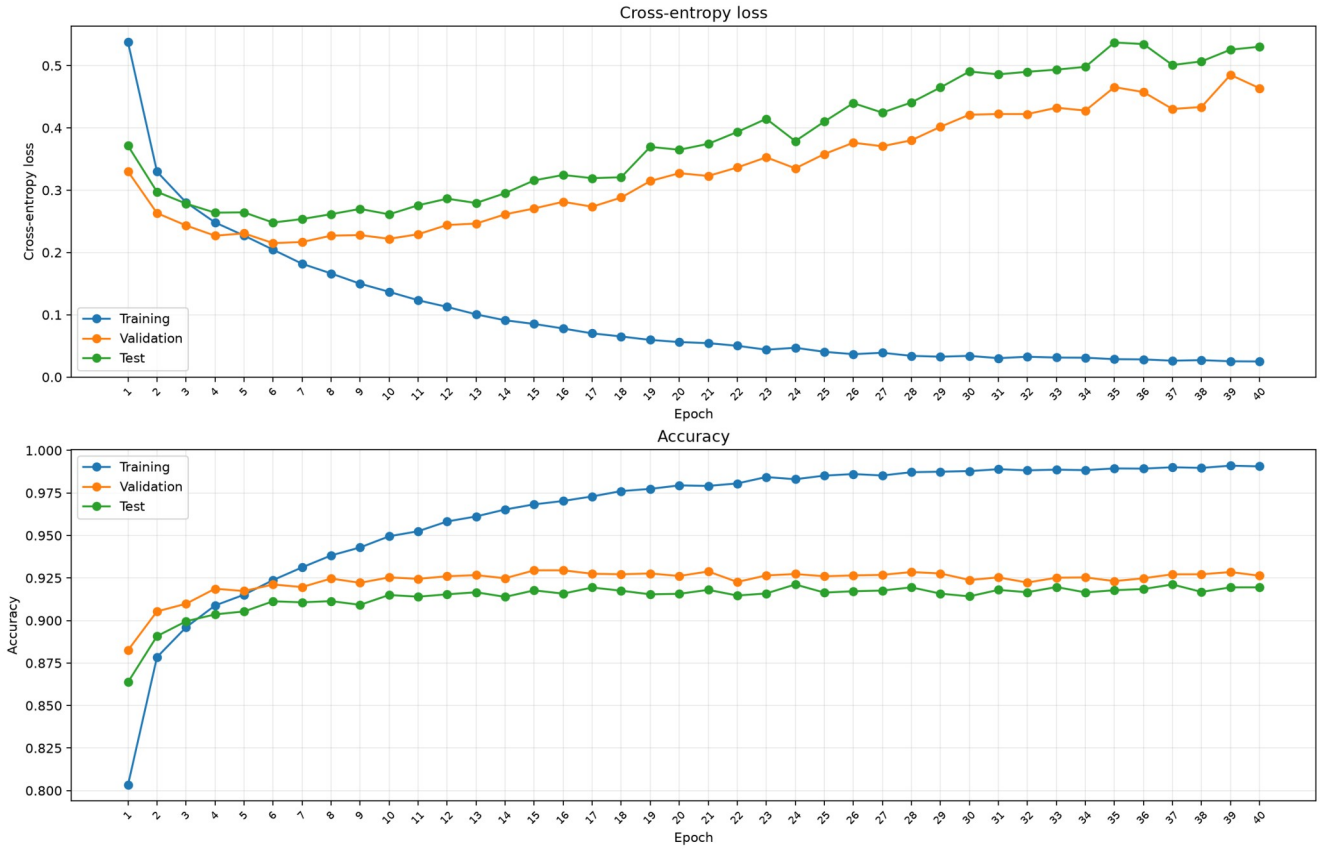


Figure 1. Loss and accuracy for training, validation and test data across all 40 epochs.

During roughly the first six epochs, all three losses fall. This means that the model improves on both the training data and unseen images. Around epoch 6, validation loss is near its lowest point, at about 0.21, while test loss is about 0.25. These values are approximate readings from the graph.

After this point, training loss continues to fall, but validation and test loss generally rise, with some short fluctuations. By epoch 40, training loss is about 0.03, validation loss about 0.46 and test loss about 0.53. The model fits the training examples more closely while its probability predictions on unseen images get worse. This is evidence of overfitting from around epoch 6.

Why accuracy does not show this as clearly

Accuracy only checks whether the class with the highest probability is correct. Cross-entropy also checks how much probability the model gives the correct class. For one image, the loss is minus the natural logarithm of that probability.

For example, suppose a shirt is wrongly predicted as a coat. If the probability of the correct shirt class falls from 0.20 to 0.02, both predictions are still wrong, so accuracy does not change. However, the loss rises from about 1.61 to 3.91. A few very confident mistakes can raise the average loss while accuracy stays similar.

Validation accuracy stays around 92 to 93 percent and test accuracy around 91 to 92 percent for much of the run. They do not show the same clear decline as the loss curves. However, training accuracy rises towards 99 percent, so the growing accuracy gap is also a warning sign. Loss gives the clearer signal here.

Test predictions by clothing class

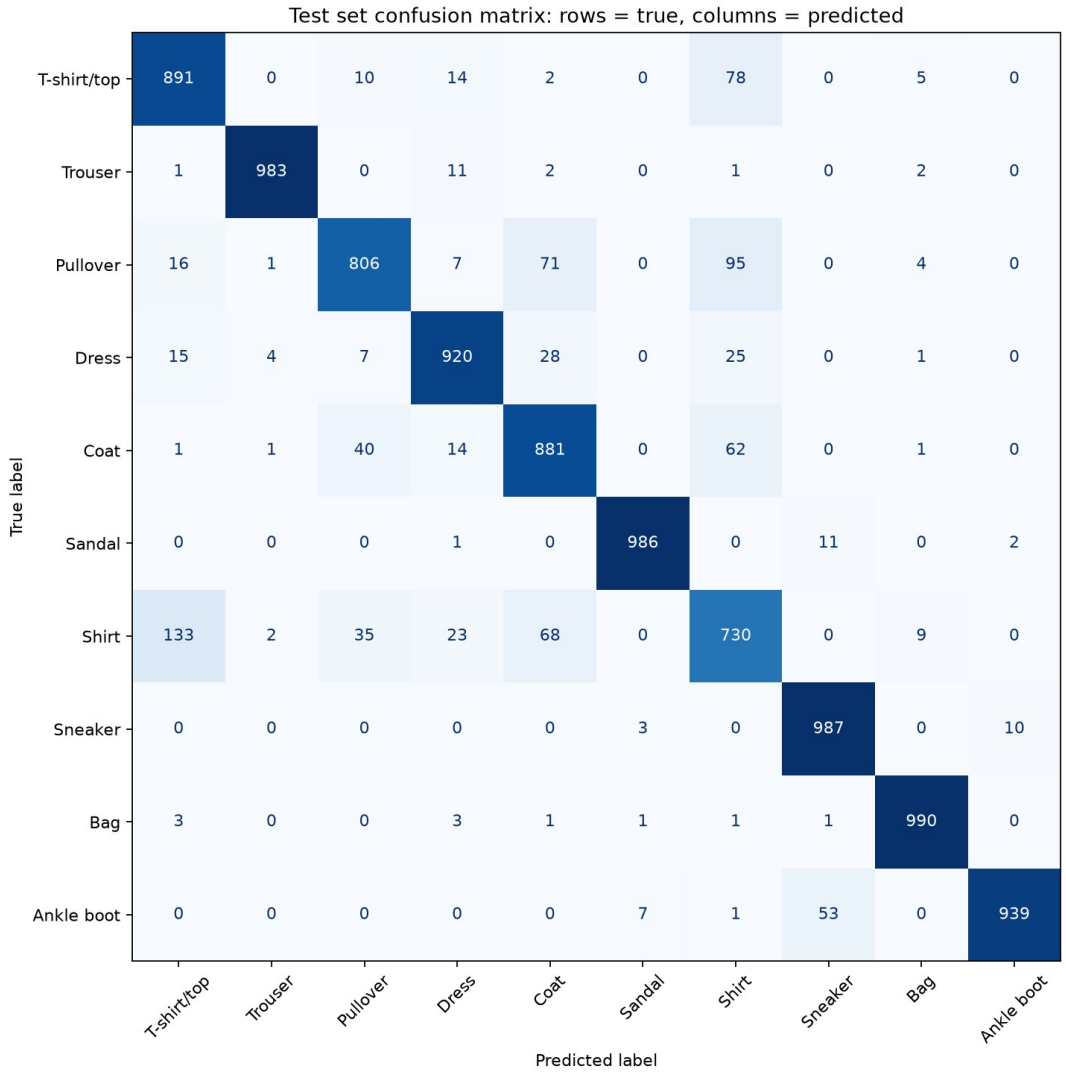


Figure 2. Test confusion matrix for the weights selected using the lowest validation loss. Rows are true classes and columns are predictions.

The diagonal contains correct predictions. Adding its values gives 9,113 correct predictions out of 10,000 test images, or 91.13 percent accuracy. This matrix describes the selected model, not necessarily the model at epoch 40.

Bags, sneakers, sandals and trousers are classified well: 990, 987, 986 and 983 correct predictions out of 1,000 for each class. Shirts are harder, with 730 correct predictions. The model predicts 133 shirts as T-shirts or tops, and 95 pullovers as shirts. Similar clothing shapes may help explain these errors.

Conclusion

The CNN learns useful patterns, but training for longer does not keep improving its predictions on unseen data. The clearest warning appears in the loss curves after about epoch 6. Accuracy alone would make this problem easier to miss. For this run, selecting the weights with the lowest validation loss is more useful than simply keeping the last epoch.

These results come from one run. They support the overfitting explanation, but they do not prove that each design choice is better than other options. The next exercise can compare this CNN with a model that adds residual connections.